Hadoop & Spark - ISEN 2025/2026

Lab: Hadoop with Docker

For this Lab you will need:

- Docker Desktop installed on your machine
- That's it! No Java, no Hadoop installation, no SSH configuration

This lab provides the same learning experience as Lab 2.1 but uses Docker containers for a cleaner, more portable setup.

Why Docker?

- **No local installation mess**: Everything runs in isolated containers
- **Reproducible environment**: Same setup on any machine
- **Easy cleanup**: Just remove the containers when done
- **Closer to production**: Real clusters often use containerized deployments

Step 0: Install Docker Desktop

If you don't have Docker Desktop installed:

1. Download from docker.com
2. Install and restart your computer if prompted
3. Launch Docker Desktop and wait for it to start

Verify installation in your terminal:

```
docker --version
docker-compose --version
```

Step 1: Create the project structure

Create a new folder for this lab and navigate to it.

Ex on linux:

```
mkdir hadoop-docker-lab
cd hadoop-docker-lab
```

Step 2: Create docker-compose.yml (1/3)

Create a file named `docker-compose.yml` with the following content:

```yaml
version: "3"

services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: namenode
    restart: always
    ports:
      - 9870:9870
      - 9000:9000
    volumes:
      - hadoop_namenode:/hadoop/dfs/name
      - ./data:/data
    environment:
      - CLUSTER_NAME=test
    env_file:
      - ./hadoop.env
```

Step 2: Create docker-compose.yml (2/3)

Continue adding to the same file:

```
  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode
    restart: always
    volumes:
      - hadoop_datanode:/hadoop/dfs/data
    environment:
      SERVICE_PRECONDITION: "namenode:9870"
    env_file:
      - ./hadoop.env

  resourcemanager:
    image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
    container_name: resourcemanager
    restart: always
    ports:
      - 8088:8088
    environment:
      SERVICE_PRECONDITION: "namenode:9000 namenode:9870 datanode:9864"
    env_file:
      - ./hadoop.env
```

Step 2: Create docker-compose.yml (3/3)

Add the remaining services and volumes:

```
  nodemanager:
    image: bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8
    container_name: nodemanager
    restart: always
    environment:
      SERVICE_PRECONDITION: "namenode:9000 namenode:9870 datanode:9864 resourcemanager:8088"
    env_file:
      - ./hadoop.env

volumes:
  hadoop_namenode:
  hadoop_datanode:
```

Step 3: Create hadoop.env

Create a file named hadoop.env with the Hadoop configuration:

```
CORE_CONF_fs_defaultFS=hdfs://namenode:9000
CORE_CONF_hadoop_http_staticuser_user=root

HDFS_CONF_dfs_webhdfs_enabled=true
HDFS_CONF_dfs_permissions_enabled=false
HDFS_CONF_dfs_replication=1

YARN_CONF_yarn_log___aggregation___enable=true
YARN_CONF_yarn_resourcemanager_hostname=resourcemanager
YARN_CONF_yarn_nodemanager_aux___services=mapreduce_shuffle

MAPRED_CONF_mapreduce_framework_name=yarn

# You may need to change memory requirements
MAPRED_CONF_mapred_child_java_opts=-Xmx1024m
MAPRED_CONF_mapreduce_map_memory_mb=1024
MAPRED_CONF_mapreduce_reduce_memory_mb=2048
MAPRED_CONF_mapreduce_map_java_opts=-Xmx768m
MAPRED_CONF_mapreduce_reduce_java_opts=-Xmx1536m

MAPRED_CONF_yarn_app_mapreduce_am_env=HADOOP_MAPRED_HOME=/opt/hadoop-3.2.1
MAPRED_CONF_mapreduce_map_env=HADOOP_MAPRED_HOME=/opt/hadoop-3.2.1
MAPRED_CONF_mapreduce_reduce_env=HADOOP_MAPRED_HOME=/opt/hadoop-3.2.1

YARN_CONF_yarn_nodemanager_resource_memory___mb=8192
YARN_CONF_yarn_scheduler_minimum___allocation___mb=512
YARN_CONF_yarn_scheduler_maximum___allocation___mb=8192
```

Step 4: Create the data folder

Create a folder to share data with the containers:

```
mkdir data
```

This folder will be mounted inside the namenode container at /data, allowing you to easily transfer files.

Step 5: Start the Hadoop cluster (1/2)

Start all containers:

```
docker-compose up -d
```

Wait about 30-60 seconds for all services to initialize.

Potential error:

```
Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:9000 -> 127.0.0.1:0: listen tcp 0.0.0.0:9000: bind:
Only one usage of each socket address (protocol/network address/port) is normally permitted.
```

-> Try changing ports (ex -> 9000 to 9001)

Step 5: Start the Hadoop cluster (2/2)

Check that all containers are running:

```
docker ps
```

You should see 4 containers: `namenode`, `datanode`, `resourcemanager`, `nodemanager`

```
ROQUETTE+7300689@FR-5CD4253HZC MINGW64 ~/source/repos/hadoop-spark-course/Docker_lab (main)
$ docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED             STATUS                     PORTS                                                                                                  NAMES
08596425383c   bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8          "/entrypoint.sh /run…"   About a minute ago  Up About a minute (healthy)  0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp, 0.0.0.0:9001->9000/tcp, [::]:9001->9000/tcp   namenode
ea9f24b6d523   bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8          "/entrypoint.sh /run…"   7 minutes ago       Up 7 minutes (healthy)     9864/tcp                                                                                               datanode
685edc07e4f5   bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8    "/entrypoint.sh /run…"   7 minutes ago       Up 7 minutes (healthy)     0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp                                                            resourcemanager
a49da4298dd4   bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8        "/entrypoint.sh /run…"   7 minutes ago       Up 7 minutes (healthy)     8042/tcp                                                                                               nodemanager
```

> Note: you can stop docker by using: `docker-compose down`

Step 6: Verify the cluster is running

Open your browser and check:

- HDFS UI: http://localhost:9870
- YARN UI: http://localhost:8088

HDFS UI

Navigate to http://localhost:9870 to see the HDFS NameNode UI:

- Check the Overview tab to see cluster summary
- Browse the file system under Utilities > Browse the file system

YARN UI

Navigate to http://localhost:8088 to see the YARN Resource Manager:

- Check Cluster > Nodes to see your nodemanager
- This is where you'll see your MapReduce jobs running

Step 7: Prepare input data

Let's create some sample text files to process. In your data folder, create a file:

Using your text editor, create `data/sample.txt`:

```
A long time ago in a galaxy far far away
It is a period of civil war
Rebel spaceships striking from a hidden base
have won their first victory against the evil Galactic Empire
```

Step 8: Connect to the namenode container

Open a shell inside the namenode container:

```
docker exec -it namenode bash
```

You are now inside the container with Hadoop pre-installed and configured!

Useful HDFS commands reference (1/2)

List files

```
hdfs dfs -ls /path/
```

Create directory

```
hdfs dfs -mkdir -p /path/to/dir
```

Upload file to HDFS

```
hdfs dfs -put localfile /hdfs/path/
```

Download file from HDFS

```
hdfs dfs -get /hdfs/path/file localfile
```

Useful HDFS commands reference (2/2)

View file content

```
hdfs dfs -cat /hdfs/path/file
```

Delete file or directory

```
hdfs dfs -rm -r /hdfs/path/
```

Check disk usage

```
hdfs dfs -du -h /path/
```

Step 9: Create HDFS directories

Inside the container, create an input directory in HDFS:

```
hdfs dfs -mkdir -p /user/root/input
```

Verify the directory was created:

```
hdfs dfs -ls /user/root/
```

Step 10: Upload data to HDFS

Copy the sample file from the shared `/data` folder to HDFS:

```
hdfs dfs -put /data/sample.txt /user/root/input/
```

Verify the file is in HDFS:

```
# List
hdfs dfs -ls /user/root/input/
# Read content
hdfs dfs -cat /user/root/input/sample.txt
```

Step 11: Run your first MapReduce job (1/2)

Run the WordCount example (a classic MapReduce demo):

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar wordcount /user/root/input /user/root/output
```

This will:

- Map: Split text into words and emit (word, 1) pairs
- Shuffle: Group by word
- Reduce: Sum the counts for each word

> Note: this may come in handy to clean output:
> `hdfs dfs -rm -r /user/root/output`

Step 11: Run your first MapReduce job (2/2)

While the job runs, you can monitor it in the YARN UI at http://localhost:8088

Look for your application in the Applications list!

Show 20 entries

ID	User	Name	Application Type	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores	Allocated Memory MB	Reserved CPU VCores	Reserved Memory MB	% of Queue	% of Clus
application_1768328664396_0001	root	word count	MAPREDUCE	default	0	Tue Jan 13 19:24:51 +0100 2026	Tue Jan 13 19:24:52 +0100 2026	N/A	RUNNING	UNDEFINED	1	1	2048	0	0	25.0	25.0

Showing 1 to 1 of 1 entries

Step 12: Check the results

View the output:

```
hdfs dfs -cat /user/root/output/*
```

You should see word counts like:

```
A       1
Empire  1
Galactic        1
It      1
Rebel   1
a       3
...
```

Step 13: Try the grep example (1/2)

Let's run the a grep example.

> grep is a command-line tool for searching text patterns in files using regular expressions.

First, let's add some XML files that are siting there:

```
hdfs dfs -mkdir /user/root/input2/
hdfs dfs -put /opt/hadoop-3.2.1/etc/hadoop/*.xml /user/root/input2/
```

Check what you just moved to the input folder:

```
hdfs dfs -ls /user/root/input2/
```

Step 13: Try the grep example (2/2)

We will try to find all instances of `dfs...` in the XML files

Run the grep MapReduce:

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar grep /user/root/input2/ /user/root/output2/ 'dfs[a-z.]+'
```

And check the result:

```
hdfs dfs -cat /user/root/output2/*
```

You should see results similar to this:

```
1       dfsadmin
1       dfs.replication
...
```

Exit and cleanup

Exit the container shell:

```
exit
```

To stop the cluster (preserves data):

```
docker-compose down
```

To stop and remove all data:

```
docker-compose down -v
```

Troubleshooting

Containers not starting?

```
docker-compose logs namenode
docker-compose logs datanode
```

Port already in use? Change the port mapping in `docker-compose.yml` (ex: `9871:9870`)

Out of memory? Increase Docker Desktop memory allocation in `hadoop.env`

To go further: Exercise 1 - Different regex patterns

Exercise 1: Try different regex patterns with grep

The grep MapReduce searches for patterns using regular expressions. Try these:

Pattern	What it finds
`yarn[a-z.]*`	All YARN-related configurations
`<[a-z]+>`	Opening XML tags
`localhost:[0-9]+`	Localhost with port numbers
`mapreduce[a-z.]*`	MapReduce configurations

Example:

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar \
  grep /user/root/input3/ /user/root/output3/ 'yarn[a-z.]*'
hdfs dfs -cat /user/root/output3/*
```

To go further: Exercise 2 - Process your own files

Exercise 2: Add and process your own files

1. On your host machine, add text files to the `data/` folder (e.g., a book, lyrics, articles)
2. Inside the namenode container, upload them to HDFS:

```
hdfs dfs -put /data/myfile.txt /user/root/input4/
```

3. Run WordCount on your new data:

```
hdfs dfs -rm -r /user/root/output  # Clean previous output
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar \
  wordcount /user/root/input4 /user/root/output4
hdfs dfs -cat /user/root/output4/* | head -20
```

To go further: Exercise 3 - Pi estimation

Exercise 3: Estimate Pi using Monte Carlo simulation

This MapReduce estimates π by randomly throwing "darts" at a square and counting how many land inside a quarter circle.

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar \
  pi <num_maps> <samples_per_map>
```

Parameter	Meaning
`num_maps`	Number of parallel map tasks (more = faster)
`samples_per_map`	Random points per map (more = more accurate)

Try different values and compare accuracy:

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar pi 10 100
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar pi 10 10000
hadoop jar /opt/hadoop-3.2.1/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.2.1.jar pi 100 10000
```

To go further: Exercise 4 - Scale up the cluster

Exercise 4: Add more DataNodes

Edit `docker-compose.yml` and duplicate the datanode service:

```yaml
  datanode2:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: datanode2
    restart: always
    volumes:
      - hadoop_datanode2:/hadoop/dfs/data
    environment:
      SERVICE_PRECONDITION: "namenode:9870"
    env_file:
      - ./hadoop.env
```

Add the new volume at the bottom:

```yaml
volumes:
  hadoop_namenode:
  hadoop_datanode:
  hadoop_datanode2:  # Add this line
```

Restart the cluster and verify at http://localhost:9870 (Datanodes tab should show 2 nodes)

This is it!

Congratulations! You've run Hadoop MapReduce using Docker containers! 🎉

You now have a portable, reproducible Hadoop environment that you can start and stop at will.